1. Complexity Table / Fast Lookup

Algorithm/Data Struct	Time	Space	Notes
Array access	$O(1)$	-	contiguous
Vec push	amortized $O(1)$	-	resize doubles
Linked list search	$O(n)$	$O(n)$	bad locality
Stack push/pop	$O(1)$	$O(n)$	Vec
Queue push/pop	$O(1)$	$O(n)$	VecDeque
Hash search/insert/delete	expected $O(1)$, worst $O(n)$	$O(n)$	$\alpha = n/m$
BST ops	$O(h)$	$O(n)$	h $\log n$ balanced, n worst
Inorder BST	$\Theta(n)$	$O(h)$	sorted output
Insertion sort	best $\Theta(n)$, worst $\Theta(n^2)$	$O(1)$	stable
Merge sort	$\Theta(n \log n)$	$\Theta(n)$	stable
Quicksort randomized	expected $\Theta(n \log n)$	$O(\log n)$	worst $\Theta(n^2)$
Counting sort	$\Theta(n+k)$	$\Theta(n+k)$	integer keys
Heapify	$O(\log n)$	recursion	assumes children heaps
Build heap	$\Theta(n)$	$O(1)$	bottom-up
Heapsort	$\Theta(n \log n)$	$O(1)$	not stable
PQ insert/extract	$O(\log n)$	$O(n)$	heap
BFS	$\Theta(V+E)$	$\Theta(V)$	unweighted shortest paths
DFS	$\Theta(V+E)$	$\Theta(V)$	topo/cycles/SCC
Topological sort	$\Theta(V+E)$	$\Theta(V)$	DAG only
Kosaraju SCC	$\Theta(V+E)$	$\Theta(V+E)$	2 DFS
Union-Find op	$O(\alpha(n))$	$O(n)$	rank+compression
Kruskal	$O(E \log E)$	$O(V)$	sort + DSU
Prim	$O(E \log V)$	$O(V+E)$	PQ
Dijkstra	$O(E \log V)$	$O(V+E)$	no negative edges
Bellman-Ford	$\Theta(VE)$	$\Theta(V)$	negative edges OK
DAG shortest path	$\Theta(V+E)$	$O(V)$	topo order
Ford-Fulkerson	$O(E\|f^*\|)$	$O(V^2+E)$	integer caps, residual
Edmonds-Karp	$O(VE^2)$	$O(V^2+E)$	BFS residual
Rod cutting	$\Theta(n^2)$	$\Theta(n)$	DP
Matrix chain	$\Theta(n^3)$	$\Theta(n^2)$	DP
LCS	$\Theta(nm)$	$\Theta(nm)$	DP
Knapsack 0/1	$\Theta(nW)$	$\Theta(nW)/\Theta(W)$	pseudo-poly
Coin change	$\Theta(W \cdot \#\text{coins})$	$\Theta(W)$	unbounded

Exam Pattern Matcher

new optimization + choices	DP: state/base/transition/order/answer/proof
local best + exchange	greedy: safe choice + induction
connect all min weight	MST: Kruskal/Prim + cut property
unweighted shortest path	BFS levels
weighted no negative	Dijkstra
negative weights / cycle	Bellman--Ford
dependencies / precedence	DAG + topological order
capacity / matching / disjoint paths	max-flow reduction
expected count	indicators + linearity
online unknown future	competitive ratio vs OPT

Exam Writeup Checklist

Algorithm: input/output, data structs, return value. **Correctness:** complete invariant/induction/exchange proof, not idea. **DP:** State, Base, Transition, Order, Answer, Runtime, Space. **Bounds:** define n, V, E, W; justify loops. **Conventions:** $<$ vs $\leq$, inclusive/exclusive, 0/1-index, reachable vs global negative cycle.

Proof Bank: Generic Templates

Loop inv. Init true before loop. Maint: one step preserves. Term: invariant + stop condition $\Rightarrow$ answer.

Induction/substitution. Claim upper/lower bound with constants. State IH for all smaller inputs. Substitute IH. Collect terms. Pick constants so inequality holds. Base cases: choose c large enough for all $n < n_0$.

Substitution trap. If $T(n) \leq c\sum_i (a_i n)^p + bn$ and $\sum_i a_i^p = 1$, then $cn^p + bn$ FAILS. No slack absorbs $+bn$. Strengthen: $T(n) \leq cn^p - dn^q$ with lower-order subtraction.

Midterm recurrence. $T(n) = T(4n/5) + T(3n/5) + an$. Guess $\Theta(n^2)$ because $(4/5)^2 + (3/5)^2 = 1$. Lower: $T(n) \geq T(4n/5) + T(3n/5) \geq c((4n/5)^2 + (3n/5)^2) = cn^2$. Upper must be strengthened: assume $T(m) \leq cm^2 - dm$ for all $m < n$. Then $T(n) \leq c(4n/5)^2 - d(4n/5) + c(3n/5)^2 - d(3n/5) + an = cn^2 - 7dn/5 + an \leq cn^2 - dn$ iff $d \geq 5a/2$. Choose d so, then choose c large for base cases.

D&C. Induct on size. Recursive calls solve smaller problems. Combine covers all optimum cases and constructs/chooses correct result.

DP State meaning. Base correct. Induct in fill order. Transition enumerates all valid first/last choices; dependencies already optimal. Optimal solution uses one enumerated choice; every candidate feasible. Therefore $dp[s]$ optimum. Write answer $= dp[\dots]$.

Greedy. Safe choice: for optimum O containing current partial solution, if chosen $x \notin O$, exchange $f \in O$ with e without worsening. Then induct.

Proof Bank: Sorting / Hash / DS

BST inorder. Induct on subtree. Left keys $<$ root, right keys $\geq$ root. IH sorts left/right. Left,root,right sorted and complete.

Rank/interval. Store $size = 1 + size(L) + size(R)$. Rank follows one search path; when going right add $size(L) + 1$. Thus $O(h)$. $[a, b] = rank_<(b) - rank_<(a)$.

Heapify. Children are heaps. Swap with larger child if violated; parent fixed. Only swapped child may violate; recurse to leaf. Cost height $O(\log n)$.

BuildHeap. Leaves are heaps. Bottom-up: children already heaps, so Heapify fixes subtree. Runtime: nodes height $h \leq n/2^{h+1}$, total $\sum_h O(h)n/2^h = O(n)$.

Sorting lower bound. Comparison sort decision tree needs $\geq n!$ leaves. Worst-case comparisons $\geq$ height $\geq \log_2(n!) = \Omega(n \log n)$.

Quicksort exp. Pair compared at most once. $X_{ij} = 1$ if ranks i, j compared. Compare iff first pivot among ranks $i..j$ is i or j, so $P(X_{ij}) = 2/(j - i + 1)$. Sum $O(n \log n)$.

Hashing. For key x, $X_y = 1[h(y) = h(x)]$. Universal hashing: $E[X_y] \leq 1/m$. Expected collisions $\leq n/m = \alpha$. Search $O(1 + \alpha)$; if $m = \Theta(n)$ then expected $O(1)$.

DSU. Union by rank attaches smaller rank under larger. Rank increases only on equal-rank merge. Path compression preserves reps and shortens future paths. With both: m ops in $O(m\alpha(n))$.

Proof Bank: Graph Traversal

BFS. Queue processed in nondecreasing distance. Discover v from u: $d[v] = d[u] + 1$. Shorter path would come from smaller-distance vertex processed earlier, so v would already be discovered.

DFS/topo. Directed cycle iff DFS has back edge to GRAY ancestor. DAG has no back edges. For every edge $u \to v$, $f[u] > f[v]$; decreasing finish time is topo order.

DFS edge facts. Parenthesis theorem: finish intervals disjoint or nested; nested iff descendant. White-path theorem: v descendant of u iff when u discovered, there is a white path $u \rightsquigarrow v$.

SCC/Kosaraju. Condensation graph is DAG. First DFS finish order puts source SCCs late. In reversed graph, DFS from highest finish cannot leave its SCC to an unvisited SCC. Each second-pass DFS outputs one SCC.

DAG SP/LP Topo order processes every predecessor before vertex. When processing u, $dist[u]$ already final over all paths ending at u; relaxing outgoing edges propagates optimum once. Runtime $\Theta(V + E)$.

Proof Bank: MST / Shortest Paths / Flow

Cut property. Let e be light edge crossing cut respecting A. Take MST $T \supseteq A$ not using e. Add e creates cycle with another crossing edge f. Since $w(e) \leq w(f)$, $T - f + e$ is MST containing $A \cup \{e\}$.

Cycle property. If MST T contains unique heaviest edge e on cycle, remove e; cycle has another edge f reconnecting cut with $w(f) < w(e)$. Then $T - e + f$ cheaper, contradiction.

Kruskal. DSU components define cut respecting chosen edges. Next accepted edge is lightest crossing some component cut. Cut property says safe. Induct until $|V| - 1$ edges.

Prim. S = vertices in tree. Heap chooses lightest edge crossing $(S, V \setminus S)$. Cut property says safe. Add one vertex; repeat.

Dijkstra. S = finalized. Pick outside x with min tentative dist. Any alternative path first crosses from S to outside at (u, v). Since $w \geq 0$, path length $\geq d[u] + w(u, v) \geq d[v] \geq d[x]$. So $d[x]$ final.

Bellman-Ford. After i passes, all shortest paths using $\leq i$ edges are correct. Induct by last edge relaxation. If no negative cycle, shortest simple path has $\leq V - 1$ edges. If pass V improves, reachable negative cycle.

Max-flow. If no augmenting path exists, let S be vertices reachable from s in residual graph. For every edge $u \in S, v \notin S$: $cf(u, v) = 0$, therefore $f(u, v) = c(u, v)$. For every edge $u \notin S, v \in S$: $cf(v, u) = 0$, therefore $f(u, v) = 0$.
Hence net flow crossing cut equals cut capacity: $|f| = c(S, T)$.
Since every flow $\leq$ every cut, flow and cut are both optimal.

Flow cert. Any flow value $\leq$ any cut capacity. Flow value F + cut capacity F proves both optimal.

Bipartite matching flow. Caps 1 force each left/right vertex used at most once. Integral capacities give integral max flow. Unit flow through $l \to r$ is matched pair; every matching gives same-size flow.

Proof Bank: Online / Random

Ski rental. Rent 1/day, buy B. Rent until paid B, then buy. If $T \leq B$, ALG$=T$=OPT. If $T > B$, ALG$\leq 2B = 2$OPT. Lower bound: adversary stops just before/after algorithm buys.

LRU paging. Cache size k.
Phase = requests containing $\leq k$ distinct pages.
New phase starts at request of $(k + 1)$-st distinct page.
LRU: $\leq k$ misses/phase.
Reason: requested page stays in cache until k other distinct pages requested.
OPT: ≥ 1 miss/completed phase.
Reason: phase + first request next phase contains $k + 1$ distinct pages; cache stores only k.
Therefore
LRU $\leq k \cdot$OPT $+ O(k)$
$\Rightarrow$
LRU is k-competitive.

2. Complexity / Asymptotics

$\log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n! \ll n^n$
$O(g) : f \leq cg$ eventually. $\Omega(g) : f \geq cg$. $\Theta(g) = O \cap \Omega$. $o(g) : f/g \to 0$. $\omega(g) : f/g \to \infty$. Stirling: $n! \sim \sqrt{2\pi n}(n/e)^n$, so $\log(n!) = \Theta(n \log n)$.

Useful sums:

$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$, $\sum_{i=0}^{\log n} n = n \log n$, $\sum_{i \geq 0} 2^{-i} = 2$, $\sum_{h \geq 0} h2^{-h} = 2$

Master Theorem:

$a/b^d < 1 \Rightarrow \Theta(n^d)$ $a/b^d = 1 \Rightarrow \Theta(n^d \log n)$ $a/b^d > 1 \Rightarrow$ recurse dominates
$T(n) = aT(n/b) + f(n)$
$d = \log_b a$
Case 1: $f(n) = O(n^{d-\varepsilon}) \implies T(n) = \Theta(n^d)$
Case 2: $f(n) = \Theta(n^d \log^k n) \implies T(n) = \Theta(n^d \log^{k+1} n)$
Case 3: $f(n) = \Omega(n^{d+\varepsilon})$ and $af(n/b) \leq cf(n)$, $c < 1 \implies T(n) = \Theta(f(n))$

Akra–Bazzi mini-case: $T(n) = \alpha T(an) + \beta T(bn) + \Theta(n^d)$, find p with $\alpha a^p + \beta b^p = 1$. $p > d \Rightarrow \Theta(n^p)$, $p = d \Rightarrow \Theta(n^d \log n)$, $p < d \Rightarrow \Theta(n^d)$. Quick compare at d: if $\alpha a^d + \beta b^d < 1$ then $\Theta(n^d)$; if $= 1$ then $\Theta(n^d \log n)$; if > 1 solve for p.

Common recurrences:

Binary search	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
Merge sort	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Max subarray D&C	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Naive matmul D&C	$T(n) = 8T(n/2) + n^2$	$\Theta(n^3)$
Strassen	$T(n) = 7T(n/2) + n^2$	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$
Heapify	$T(n) \leq T(2n/3) + 1$	$O(\log n)$
Quicksort worst	$T(n) = T(n-1) + n$	$\Theta(n^2)$
Quicksort balanced	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Bellman-Ford	$(V-1)E$ relaxations	$\Theta(VE)$
BFS/DFS	visit V, scan E	$\Theta(V + E)$

3. Basic Data Structures

Stack / Queue / Linked List

Stack	push/pop/top	$O(1)$, array end, underflow if empty
Queue	enqueue/dequeue	$O(1)$ with circular array / deque
Linked list	search/insert/delete	search $O(n)$, known-node delete $O(1)$

Doubly linked list insert head: `x.next=head; if head!=NIL head.prev=x; head=x; x.prev=NIL`. Delete known node: reconnect `prev.next` and `next.prev`; finding node still $O(n)$.

Hash Tables

Idea: bucket $h(key) \bmod m$. **Collisions:** chaining / probing. **Load factor** $\alpha = n/m$.

insert	$\mathbb{E}[O(1)]$	worst $O(n)$
search	$\mathbb{E}[O(1+\alpha)]$	worst $O(n)$ $\quad m = \Theta(n) \Rightarrow \alpha = \Theta(1)$
delete	$\mathbb{E}[O(1)]$	worst $O(n)$

Traps: HashMap/HashSet $O(1)$ is expected/amortized, not worst-case. Bad hashing / all collisions $\Rightarrow O(n)$.

Binary Search Trees

Binary search (sorted array):

```
fn binary_search(a: Vec<_>,x){
 l=0; r=a.len();
 while l<r {
   m=(l+r)/2;
   if a[m]==x { return Some(m); }
   if a[m]<x  { l=m+1; }
   else       { r=m; }
 }
 None
}
//lower_bound = first >= x: First position where the value is >= x.
//upper_bound = first > x: First position where the value is > x.
```

BST property:

$$\forall y \in L(x) : y.key < x.key, \quad \forall y \in R(x) : y.key \geq x.key.$$

search/insert/delete	$O(h)$
min/max/successor/predecessor	$O(h)$
inorder traversal	$\Theta(n)$
balanced tree	$h = \Theta(\log n)$
degenerate tree	$h = \Theta(n)$

```
search(x,k):
  while x != NIL and x.key != k:
    if k < x.key: x = x.left
    else:         x = x.right
  return x

min(x): while x.left  != NIL: x = x.left;  return x
max(x): while x.right != NIL: x = x.right; return x

successor(x):
  if x.right != NIL: return min(x.right)
  y = x.parent
  while y != NIL and x == y.right:
    x = y
    y = y.parent
  return y
```

```
insert(T,z):
 y=NIL; x=T.root
 while x!=NIL:
   y=x
   if z.key<x.key: x=x.left else: x=x.right
 z.parent=y
 if y==NIL: T.root=z
 else if z.key<y.key: y.left=z
 else: y.right=z

transplant(T,u,v):
 if u.parent==NIL: T.root=v
 else if u==u.parent.left: u.parent.left=v
 else: u.parent.right=v
 if v!=NIL: v.parent=u.parent

delete(T,z):
 if z.left==NIL: transplant(T,z,z.right)
 else if z.right==NIL: transplant(T,z,z.left)
 else:
   y=min(z.right)
   if y.parent!=z:
     transplant(T,y,y.right);
     y.right=z.right; y.right.parent=y
   transplant(T,z,y);
   y.left=z.left; y.left.parent=y
```

Delete cases: no left, no right, or replace by successor. All $O(h)$.

Traversals:

```
inorder(x):  if x!=NIL: inorder(x.left); visit(x); inorder(x.right)
preorder(x): if x!=NIL: visit(x); preorder(x.left); preorder(x.right)
postorder(x):if x!=NIL: postorder(x.left); postorder(x.right); visit(x)
```

inorder(BST) = sorted order

Augmented BST / rank queries: Each node stores

$$size(x) = 1 + size(x.left) + size(x.right).$$

$$rank_<(x) = \#\{k : k < x\}$$
$$rank_\leq(x) = \#\{k : k \leq x\}$$

$[a, b]$	$rank_\leq(b) - rank_<(a)$
$(a, b]$	$rank_\leq(b) - rank_\leq(a)$
$[a, b)$	$rank_<(b) - rank_<(a)$
(a, b)	$rank_<(b) - rank_\leq(a)$

Rank query follows one root-to-leaf path $\Rightarrow O(h)$.

4. Sorting Algorithms

Insertion Sort

In-place, stable. Invariant: before iteration j, prefix $A[0..j)$ sorted.

```
fn insertion_sort(a: &mut Vec<i32>) {
  for j in 1..a.len() {
    let key = a[j];
    let mut i = j;
    while i > 0 && a[i-1] > key {
      a[i] = a[i-1]; i -= 1;
    }
    a[i] = key;
  }
}
```

Best $\Theta(n)$, worst/avg $\Theta(n^2)$, space $O(1)$.

Merge Sort

Stable, simple version uses $\Theta(n)$ temp.

```
fn merge_sort(a,l,r){
 if r-l <= 1 { return; }
 let m=(l+r)/2;
 merge_sort(a,l,m);
 merge_sort(a,m,r);
 merge(a,l,m,r);
}
```

```
fn merge(a,l,m,r){
  let (mut i,mut j)=(l,m); let mut t=Vec::new();
  while i<m && j<r {
    if a[i] <= a[j] { t.push(a[i]); i+=1; }
    else { t.push(a[j]); j+=1; }
  }
  while i<m { t.push(a[i]); i+=1; }
  while j<r { t.push(a[j]); j+=1; }
  for k in 0..t.len(){ a[l+k]=t[k]; }
}
```

$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Quicksort

In-place. Partition invariant: $A[l..i] \leq p$, $A[i+1..j] > p$, unknown after j, pivot at r.

```
fn partition(a,l,r)->usize{
 let p=a[r]; let mut i=l;
 for j in l..r {
   if a[j] <= p { a.swap(i,j); i+=1; }
 }
 a.swap(i,r); i
}
```

```
fn quicksort(a,l,r){
 if l>=r { return; }
 let q=partition(a,l,r);
 if q>0 { quicksort(a,l,q-1); }
 quicksort(a,q+1,r);
}
```

Balanced/expected randomized $\Theta(n \log n)$, worst $\Theta(n^2)$, stack expected $O(\log n)$ worst $O(n)$.

Counting Sort / Linear Sorting

Integer keys $0..k$, stable with prefix sums, not comparison-based.

```
fn counting_sort(a:&Vec<usize>, k:usize)->Vec<usize>{
 let mut c=vec![0;k+1]; let mut out=vec![0;a.len()];
 for &x in a { c[x]+=1; }
 for i in 1..=k { c[i]+=c[i-1]; }
 for idx in (0..a.len()).rev(){
   let x=a[idx]; c[x]-=1; out[c[x]]=x;
 }
 out
}
```

$\Theta(n + k)$ time/space. Stable with prefix sums. Needs bounded integer keys.

5. Heaps / Priority Queues / Heapsort

Max-Heapify

Assume children subtrees are heaps; only $A[i]$ may violate. 0-index:

```
fn max_heapify(a:&mut Vec<i32>, i:usize, heap_size:usize){
 let (l,r)=(2*i+1,2*i+2); let mut largest=i;
 if l<heap_size && a[l]>a[largest] { largest=l; }
 if r<heap_size && a[r]>a[largest] { largest=r; }
 if largest!=i { a.swap(i,largest); max_heapify(a,largest,heap_size); }
}
```

$O(height(i)) \leq O(\log n)$. Trap: during heapsort, heap_size $\neq$ a.len().

Build Max Heap

```
fn build_max_heap(a:&mut Vec<i32>){
 let n=a.len();
 for i in (0..(n/2)).rev(){ max_heapify(a,i,n); }
}
```

Tight: nodes height $h \leq n/2^{h+1}$, cost $O(h)$, so $\sum_h nh/2^h = O(n)$; hence BuildHeap $\Theta(n)$, not $\Theta(n \log n)$.

Heapsort

```
fn heapsort(a:&mut Vec<i32>){
 build_max_heap(a); let mut h=a.len();
 while h>1 { a.swap(0,h-1); h-=1; max_heapify(a,0,h); }
}
```

Build $\Theta(n) + n$ extracts $O(\log n) = \Theta(n \log n)$, space $O(1)$, not stable.

Priority Queue

```
fn heap_extract_max(a:&mut Vec<i32>)->Option<i32>{
 if a.is_empty(){return None;}
 let ans=a[0]; let last=a.pop().unwrap();
 if !a.is_empty(){ a[0]=last; max_heapify(a,0,a.len()); }
 Some(ans)
}
fn heap_increase_key(a:&mut Vec<i32>, mut i:usize, key:i32){
 if key<a[i]{return;} a[i]=key;
 while i>0 { let p=(i-1)/2; if a[p]>=a[i]{break;} a.swap(i,p); i=p; }
}
fn max_heap_insert(a:&mut Vec<i32>, key:i32){
 a.push(i32::MIN); heap_increase_key(a,a.len()-1,key);
}
```

Std min-heap: `BinaryHeap<Reverse<(dist,node)>>`. Ops insert/extract/increase-key $O(\log n)$.

6. Divide & Conquer Algorithms

Maximum Sum Subarray

Problem:
Input A[0..n) Output max sum contiguous nonempty subarray
D&C idea:
best subarray is:

- **entirely left**
- **entirely right**
- **crosses middle**

```
fn max_crossing(a,l,m,r){
 // left suffix [l,m)
 left_sum=-INF;
 sum=0;
 for i in (l..m).rev(){
   sum+=a[i];
   left_sum=max(left_sum,sum);
 }
 // right prefix [m,r)
 right_sum=-INF;
 sum=0;
 for j in m..r {
   sum+=a[j];
   right_sum=max(right_sum,sum);
 }
 left_sum+right_sum
}
```

```
fn max_subarray(a,l,r){
 if r-l==1 {return a[l]}
 m=(l+r)/2;
 left=max_subarray(a,l,m);
 right=max_subarray(a,m,r);
 cross=max_crossing(a,l,m,r);
 max(left,max(right,cross))
}
```

Complexity:
$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$
Linear Kadane:

```
fn kadane(a: &Vec<i32>) -> i32 {
    let mut best: i32 = a[0];
    let mut cur: i32 = a[0];

    for i in 1..a.len() {
        cur = a[i].max(cur + a[i]);
        best = best.max(cur);
    }

    best
}
```

Kadane invariant:

$$\text{cur}_i = \max(A_i, \text{cur}_{i-1} + A_i), \qquad \text{best}_i = \max(\text{best}_{i-1}, \text{cur}_i)$$

Matrix Multiplication

Naive:

```
for i in 0..n {
    for j in 0..n {
        for k in 0..n {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

Complexity:
$\Theta(n^3)$, $space \Theta(n^2)$
D&C block formula (8 mults):

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \qquad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21} \qquad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

$T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$.
Strassen (7 mults):

$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$ $\quad M_2 = (A_{21} + A_{22})B_{11}$
$M_3 = A_{11}(B_{12} - B_{22})$ $\quad M_4 = A_{22}(B_{21} - B_{11})$
$M_5 = (A_{11} + A_{12})B_{22}$ $\quad M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$
$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$
$C_{11} = M_1 + M_4 - M_5 + M_7$ $\quad C_{12} = M_3 + M_5$
$C_{21} = M_2 + M_4$ $\quad C_{22} = M_1 - M_2 + M_3 + M_6$

```
fn strassen(A,B,n){
 if n==1 { return A*B }
 split A,B into A11..A22,B11..B22;
 M1..M7 = strassen(formulas above);
 C11=M1+M4-M5+M7; C12=M3+M5;
 C21=M2+M4;       C22=M1-M2+M3+M6;
 combine(C11,C12,C21,C22)
}
```

$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

7. Dynamic Programming

DP Meta-template

State/base/transition/order/answer/runtime/reconstruct.

```
dp[s] = opt over valid choices c:
        F(s,c, dp[next(s,c)])
```

Always state: state/base/transition/order/answer/runtime/reconstruct.

```
choice[s] = argopt choice used in dp[s]
reconstruct:
while state != terminal:
    output choice[state]
    state = next(state, choice[state])
```

Rod Cutting

revenue$[\ell]$ = best revenue length ℓ, first$[\ell]$ = first cut.

$$\text{revenue}[0] = 0, \qquad \text{revenue}[\ell] = \max_{1 \leq c \leq \ell} \{\text{price}[c] + \text{revenue}[\ell - c]\}.$$

```
fn rod_cut(price:Vec<i32>, n:usize){
 revenue=vec![0;n+1]; first=vec![0;n+1];
 for length in 1..=n {revenue[length]=-INF;
   for cut in 1..=length {
     candidate=price[cut]+revenue[length-cut];
     if candidate>revenue[length] {
       revenue[length]=candidate; first[length]=cut
     }
   }
 }
 (revenue,first)
}
```

Reconstruct: output `first[length]`, set `length-=first[length]`. Runtime $\Theta(n^2)$.

Matrix Chain Multiplication

Matrix i: $p_i \times p_{i+1}$. $dp[i][j]$ **= min cost for** $A_i \cdots A_j$.

$$dp[i][i] = 0, \qquad dp[i][j] = \min_{i \le k < j} \{dp[i][k] + dp[k+1][j] + p_i p_{k+1} p_{j+1}\}.$$

```
fn matrix_chain(p:Vec<usize>){     // optimal parenthesization
  n=p.len()-1; dp=vec![vec![0;n];n]; split=vec![vec![0;n];n];
  for len in 2..=n {for i in 0..=n-len {
    j=i+len-1; dp[i][j]=INF;
    for k in i..j {
      cost=dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1];
      if cost<dp[i][j] {dp[i][j]=cost; split[i][j]=k}
    }
  }}
  (dp,split)
}
```

Increasing chain length. $\Theta(n^2)$ **states** $\times \Theta(n)$ **splits** $= \Theta(n^3)$.

Optimal BST

Sorted keys K_i **with successful probs** p_i**; optional unsuccessful** q_i. $e[i][j]$ **= min expected cost for subtree** $i..j$, $w[i][j]$ **= prob mass,** $root[i][j]$ **= best root.**

$$e[i][i-1] = 0, \quad w[i][i-1] = q_{i-1},$$
$$e[i][j] = \min_{i \le r \le j} \{e[i][r-1] + e[r+1][j] + w[i][j]\}$$

```
fn optimal_bst(p,q,n){
  for i in 1..=n+1 {e[i][i-1]=0; w[i][i-1]=q[i-1]}
  for len in 1..=n {for i in 1..=n-len+1 {
    j=i+len-1; e[i][j]=INF; w[i][j]=w[i][j-1]+p[j]+q[j];
    for r in i..=j {
      cost=e[i][r-1]+e[r+1][j]+w[i][j];
      if cost<e[i][j] {e[i][j]=cost; root[i][j]=r}
    }
  }}
  (e,root)
}
```

Runtime $\Theta(n^3)$**, space** $\Theta(n^2)$.

Longest Common Subsequence

$dp[i][j]$ **= LCS length of** $A[0..i)$, $B[0..j)$.

$$dp[0][j] = dp[i][0] = 0,$$
$$dp[i][j] = \begin{cases} 1 + dp[i-1][j-1] & A_{i-1} = B_{j-1}, \\ \max(dp[i-1][j],\, dp[i][j-1]) & A_{i-1} \neq B_{j-1} \end{cases}$$

```
fn lcs(a:Vec<char>, b:Vec<char>){
  n=a.len(); m=b.len(); dp=vec![vec![0;m+1];n+1];
  for i in 1..=n {for j in 1..=m {
    dp[i][j]=if a[i-1]==b[j-1] {1+dp[i-1][j-1]}
             else {dp[i-1][j].max(dp[i][j-1])}
  }}
  dp[n][m]
}
fn lcs_reconstruct(a:Vec<char>, b:Vec<char>, dp:Vec<Vec<usize>>){
  i=a.len(); j=b.len(); ans=vec![];
  while i>0 && j>0 {
    if a[i-1]==b[j-1] {ans.push(a[i-1]); i-=1; j-=1;
    else if dp[i-1][j]>=dp[i][j-1] {i-=1}
    else {j-=1}
  }
  ans.reverse(); ans
}
```

$\Theta(nm)$ **time,** $\Theta(nm)$ **or rolling** $\Theta(m)$ **space. Equal → diagonal+1; unequal → max(up,left).**

Knapsack 0/1

$dp[i][c]$ **= best value using first** i **items, capacity** c.

$$dp[i][c] = \max\left(dp[i-1][c],\ v_{i-1} + dp[i-1][c - w_{i-1}]\right) \quad (w_{i-1} \le c).$$

```
fn knapsack(w:Vec<usize>, v:Vec<i32>, cap:usize){ // choose item at most once
  n=w.len(); dp=vec![vec![0;cap+1];n+1];
  for i in 1..=n {for c in 0..=cap {
    dp[i][c]=dp[i-1][c];
    if w[i-1]<=c {
      dp[i][c]=dp[i][c].max(v[i-1]+dp[i-1][c-w[i-1]])
    }
  }}
  dp[n][cap]
}
// 1D 0/1: capacity descending. Unbounded: capacity ascending.
```

$\Theta(nW)$ **pseudo-polynomial.**

Coin Change / Change-Making

$$mincoins[0] = 0, \qquad mincoins[x] = \min_{coin \le x} \{1 + mincoins[x - coin]\}.$$

```
fn coin_change(coins:Vec<usize>, target:usize){ // unlimited coins
  mincoins=vec![INF;target+1]; mincoins[0]=0;   // min number of coins
  for amount in 1..=target {for coin in coins {
    if coin<=amount {
      mincoins[amount]=mincoins[amount].min(1+mincoins[amount-coin])
    }
  }}
  mincoins[target]
}
```

Runtime $\Theta(target \cdot \#coins)$.

Longest Increasing Subsequence

Quadratic:

$$dp[i] = 1 + \max\{dp[j] : j < i \wedge a_j < a_i\}, \qquad \text{answer} = \max_i dp[i].$$

```
fn lis_n2(a:Vec<i32>){
  dp=vec![1;a.len()];
  for i in 0..a.len(){for j in 0..i{
    if a[j]<a[i]{dp[i]=dp[i].max(1+dp[j])}
  }}
  dp.max(dp)
}
fn lis_nlogn(a:Vec<i32>){ // tails[len-1] = min tail
  tails=vec![];
  for x in a {
    l=0; r=tails.len();
    while l<r {m=(l+r)/2; if tails[m]<x {l=m+1}else{r=m}}
    if l==tails.len(){tails.push(x)}else{tails[l]=x}
  }
  tails.len()
}
```

DAG Longest Path

Input `adj: WGraph`. Topological order; max-relaxation; negative weights OK because no cycles.

```
fn dag_longest(adj:WGraph, source:usize){
  n=adj.len(); dist=vec![-INF;n]; parent=vec![None;n]; dist[source]=0;
  for u in topo(adj) {if dist[u]!=-INF {for (v,w) in adj[u] {
    if dist[u]+w>dist[v] {dist[v]=dist[u]+w; parent[v]=Some(u)}
  }}}
  dist
}
```

Runtime $\Theta(V + E)$.

8. Graph Representation / Types

Use these names everywhere:

```
type Graph  = Vec<Vec<usize>>;       // adj[u] = neighbors v
type WGraph = Vec<Vec<(usize,i32)>>;  // adj[u] = (v,weight)
struct Edge { u:usize, v:usize, w:i32 }
type EdgeList = Vec<Edge>;
```

adj list	**space** $\Theta(V + E)$	**scan neighbors** $\Theta(\deg u)$
adj matrix	**space** $\Theta(V^2)$	**edge lookup** $\Theta(1)$**, scan** $\Theta(V)$
edge list	**space** $\Theta(E)$	**Bellman-Ford/Kruskal scans all edges**

9. BFS

Input `adj: Graph`. Unweighted shortest paths / levels / components / bipartite. Runtime $\Theta(V + E)$.

```
fn bfs(adj:Graph, source:usize){
  n=adj.len(); // shortest path in unweighted graph
  dist=vec![INF;n]; // levels, components
  parent=vec![None;n];
  queue=VecDeque::new();
  dist[source]=0;
  queue.push_back(source);

  while let Some(u)=queue.pop_front(){
    for v in adj[u] {
      if dist[v]==INF {
        dist[v]=dist[u]+1;
        parent[v]=Some(u);
        queue.push_back(v);
      }
    }
  }
  (dist,parent)
}
```

```
fn bipartite(adj:Graph){          // 2-coloring
  n=adj.len(); color=vec![-1;n];  // false iff odd cycle
  queue=VecDeque::new();
  for s in 0..n {
    if color[s]!=-1 {continue}
    color[s]=0; queue.push_back(s);
    while let Some(u)=queue.pop_front(){
      for v in adj[u] {
        if color[v]==-1 {
          color[v]=1-color[u];
          queue.push_back(v);
        }
        else if color[v]==color[u] {return false}
      }
    }
  }
  true
}
```

Invariant: queue processed in nondecreasing distance. First discovery gives shortest unweighted distance. If a shorter path to v existed, its predecessor would have been processed earlier, so v would already be discovered.
Uses: unweighted shortest path, levels, connected components, bipartite test.
Path reconstruction: follow parent$[t]$ backward until source, then reverse.

10. DFS / Topological Sort / SCC

Input `adj: Graph`. Colors: WHITE/GRAY/BLACK. Runtime $\Theta(V + E)$.

```
fn dfs(adj:Graph){  // topo, cycle detection, SCC
  n=adj.len(); color=vec![WHITE;n]; parent=vec![None;n]; time=0;
  fn visit(u:usize){
    time+=1; discover[u]=time; color[u]=GRAY;
    for v in adj[u] {
      if color[v]==WHITE {parent[v]=Some(u); visit(v)}
    }
    color[u]=BLACK; time+=1; finish[u]=time;
  }
  for u in 0..n {if color[u]==WHITE {visit(u)}}
}
```

Directed edges: tree = white target; back = gray ancestor; forward/cross = black. Undirected: only tree/back; cycle test = visited neighbor $\neq$ **parent.**

$d[u] < d[v] < f[v] < f[u]$	**tree/forward**	v **descendant of** u
$d[v] < d[u] < f[u] < f[v]$	**back**	v **ancestor, cycle in directed**
$f[v] < d[u]$ **or** $f[u] < d[v]$	**cross**	**disjoint subtrees**

Directed cycle detection: edge to GRAY vertex ⇔ back edge ⇔ directed cycle. Edge to BLACK does not prove a cycle.

Topological Sort

DAG order: every edge $u \to v$ **has** u **before** v. **DFS = decreasing finish times.**

```
fn topo_dfs(adj:Graph){
  n=adj.len(); seen=vec![false;n]; order=vec![];
  fn visit(u:usize){
    seen[u]=true;
    for v in adj[u]{if !seen[v]{visit(v)}}
    order.push(u)
  }
  for u in 0..n {if !seen[u]{visit(u)}}
  order.reverse(); order
}
```

```
fn topo_kahn(adj:Graph){
  n=adj.len(); indeg=vec![0;n];
  queue=VecDeque::new(); order=vec![];
  for u in 0..n {for v in adj[u]{indeg[v]+=1}}
  for u in 0..n {if indeg[u]==0 {queue.push_back(u)}}
  while let Some(u)=queue.pop_front(){
    order.push(u);
    for v in adj[u]{
      indeg[v]-=1;
      if indeg[v]==0{queue.push_back(v)}
    }
  }
  if order.len()==n {Some(order)} else {None}
}
```

Kahn returns `None` iff not all vertices are output.

Strongly Connected Components

SCC: $u \iff v$. **Kosaraju = DFS finish order, reverse graph, DFS in decreasing finish order.**

```
fn kosaraju(adj:Graph){  // SCC. DFS order -> reverse g
  order=dfs_finish_order(adj); rev=reverse_graph(adj);
  seen=vec![false;n]; comps=[];
  for source in order.rev() {
    if seen[source]{continue}
    comp=[]; stack=[source]; seen[source]=true;
    while let Some(u)=stack.pop(){
      comp.push(u);
      for v in rev[u]{
        if !seen[v]{seen[v]=true; stack.push(v)}
      }
    }
    comps.push(comp)
  }
  comps
}
```

Runtime $\Theta(V + E)$.

11. Union-Find / Disjoint Sets

Forest of trees. parent$[x]$ **points upward. Root is representative.** rank$[x]$ **upper-bounds height.**

```
struct DSU {                // connectivity
    parent: Vec<usize>,     // cycle detection
    rank: Vec<usize>,
}

impl DSU{
  fn new(n){DSU{parent:(0..n).collect(), rank:vec![0;n]}}
  fn find(&mut self,x){
    if self.parent[x]!=x {self.parent[x]=self.find(self.parent[x])}
    self.parent[x]
  }
  fn union(&mut self,x,y){
    let (mut rx,mut ry)=(self.find(x),self.find(y));
    if rx==ry {return false}
    if self.rank[rx] < self.rank[ry] { swap(rx,ry) }
    self.parent[ry]=rx;
    if self.rank[rx]==self.rank[ry] {
      self.rank[rx]+=1;
    }
    true
  }
}
```

Complexities:

- **linked list weighted union:** $O(m + n \log n)$
- **forest + union by rank + path compression:** $O(m\alpha(n))$

$\alpha(n) \le 5$ **in practice.**
Use cases:
Connected components:

```
let mut dsu: DSU = DSU::new(n);

for (u, v) in edges {
    dsu.union(u, v);
}

if dsu.find(a) == dsu.find(b) {
    // same component
}
```

Cycle detection (undirected):

```
let mut dsu: DSU = DSU::new(n);

for (u, v) in edges {
    if !dsu.union(u, v) {
        // cycle
    }
}
```

12. Minimum Spanning Trees

Chooser

sparse graph	**Kruskal often easiest**	
dense graph	**Prim often easier**	**tree: connected + acyclic connected graph: tree ⇔**
disconnected graph	**forest, no MST**	**|E| = |V| -1**

MST: spanning tree with min total edge weight

Cut / Cycle Properties

Cut $(S, V - S)$**: edge crosses cut iff one endpoint in** S**, one outside. Light edge = minimum-weight crossing edge.**
Cut property: if cut respects chosen edge set A**, then a light crossing edge is safe.**
Cycle property: for any cycle, a heaviest edge on that cycle is excluded from some MST. If weights distinct, the heaviest edge is in no MST.

Kruskal

Data structures:

```
struct Edge {
    u,
    v,
    w: i32,
}

let mut edges: Vec<Edge> = ...;
let mut dsu: DSU = DSU::new(n);
let mut mst: Vec<Edge> = Vec::new();
let mut total: i32 = 0;
```

Algorithm:

```
fn kruskal(n,edges: Vec<Edge>){ // MST
  sort_by_weight(edges); dsu=DSU::new(n); total=0; mst=[];
  for e in edges {              // sort edges + DSU
    if dsu.union(e.u,e.v) {
      total+=e.w; mst.push(e);
      if mst.len()==n-1 {break}
    }
  }
  (total,mst)
}
```

Complexity: sort: $O(E \log E)$ **DSU:** $O(E\alpha(V))$ **total:** $O(E \log E)$
Output condition: if `mst.len()==n-1`, it is an MST; otherwise graph disconnected and result is a forest.
Traps:

- **If graph disconnected → MST does not exist; returns forest.**
- **Need sort by weight increasing.**
- **Skip edge if endpoints already connected.**

Prim

```
let mut adj: Vec<Vec<(usize, i32)>> = vec![Vec::new(); n];
let mut used: Vec<bool> = vec![false; n];
let mut heap: BinaryHeap<Reverse<(i32, usize, usize)>> = BinaryHeap::new();
// (weight, vertex, parent)

fn prim(adj,s){
  heap.push((0,s)); total=0; cnt=0;     // MST
  while let Some((w,u))=heap.pop_min(){ // min crossing edge
    if used[u] {continue}
    used[u]=true; total+=w; cnt+=1;
    for (v,w2) in adj[u] {
      if !used[v] {heap.push((w2,v))}
    }
  }
  if cnt!=n {/* disconnected */}
  total
}
```

Complexity:

- **Binary heap lazy:** $O(E \log E)$**; for simple graphs** $E \le V^2$**, so** $O(E \log V)$.

Output condition: if `cnt==n`, `total` is MST weight; otherwise graph disconnected.

13. Shortest Paths

Chooser

unweighted	**BFS**
DAG	**topo DP**
weights ≥ 0	**Dijkstra**
negative edges	**Bellman-Ford**

Relaxation

For edge (u, v, w)**:**

$$\text{dist}[u] \neq \text{INF} \wedge \text{dist}[v] > \text{dist}[u] + w \Rightarrow \text{dist}[v] = \text{dist}[u] + w, \quad \text{parent}[v] = u.$$

Dijkstra

```
fn dijkstra(adj:WGraph, source:usize){ // all weights >= 0
  let n=adj.len();
  let mut dist=vec![INF;n];
  let mut parent=vec![None;n];
  let mut heap=MinHeap::<(i32,usize)>::new();

  dist[source]=0;
  heap.push((0,source));

  while let Some((du,u))=heap.pop(){
    if du!=dist[u] {continue} // stale heap entry

    for (v,w) in adj[u] {
      if dist[v]>du+w {
        dist[v]=du+w;
        parent[v]=Some(u);
        heap.push((dist[v],v));
      }
    }
  }
  dist
}
```

Runtime $O((V + E) \log V)$**. Requires all weights** ≥ 0**; negative edges break it.**

Bellman-Ford

Input `edges: EdgeList`. Negative edges OK; detects reachable negative cycles.

```
fn bellman_ford(n:usize, edges:EdgeList, source:usize){
  let mut dist=vec![INF;n];    // SSSP
  let mut parent=vec![None;n];  // negative edges OK
  dist[source]=0;              // detects reachable neg cycle

  for _ in 0..n-1 {
    let mut changed=false;
    for edge in edges {
      if dist[edge.u]!=INF
          && dist[edge.v]>dist[edge.u]+edge.w {
        dist[edge.v]=dist[edge.u]+edge.w;
        parent[edge.v]=Some(edge.u);
        changed=true;
      }
    }
    if !changed {break}
  }
```

```
  for edge in edges {
    if dist[edge.u]!=INF
        && dist[edge.v]>dist[edge.u]+edge.w {
      return None;
    }
  }
  Some(dist)
}
```

Runtime $\Theta(VE)$**. Returns `None` iff a reachable negative cycle is detected. Negative cycle not reachable from source: Bellman-Ford still returns shortest paths. Only improvements after pass V reachable from source matter.**

Shortest Paths in DAG

```
fn dag_shortest(adj:WGraph, source:usize){
  let n=adj.len();
  let mut dist=vec![INF;n];
  let mut parent=vec![None;n];

  dist[source]=0;

  for u in topo(adj) {
    if dist[u]==INF {continue}
```

```
    for (v,w) in adj[u] {    // topological order.
      if dist[v]>dist[u]+w {  // one relaxation per edge
        dist[v]=dist[u]+w;   // negative weights ok
        parent[v]=Some(u);
      }
    }
  }
  dist
}
```

Runtime $\Theta(V + E)$.

14. Flow Networks

Directed $G = (V, E)$**, source** s**, sink** t**, capacities** $c(u, v) \ge 0$.

$$0 \le f(u, v) \le c(u, v), \qquad \forall u \neq s, t : \sum_v f(v, u) = \sum_v f(u, v),$$
$$|f| = \sum_v f(s, v) - \sum_v f(v, s).$$

Augmenting path: $s \to t$ **path in residual graph; bottleneck** $= \min c_f(e)$ **on path.**
Flow reduction recipe: 1. What is being selected? 2. Capacity enforcing usage limit. 3. Source/sink edges enforce constraints. 4. Flow value = objective. 5. Integral capacities => integral solution.

Ford-Fulkerson / Edmonds-Karp

Types: `cap[u][v]` residual capacity matrix; `adj: Graph` stores forward+reverse residual neighbors.

```
fn add_edge(u:usize,v:usize,c:i32){
  adj[u].push(v); adj[v].push(u); cap[u][v]+=c
}
fn bfs_path(source:usize, sink:usize){
  parent.fill(None); parent[source]=Some(source);
  queue.push_back((source,INF));
  while let Some((u,flow))=queue.pop_front(){
    for v in adj[u]{
      if parent[v].is_none() && cap[u][v]>0 {
        parent[v]=Some(u);
        pushed=flow.min(cap[u][v]);
        if v==sink {return pushed}
        queue.push_back((v,pushed))
      }
    }
  }
  0
}
```

```
fn max_flow(source:usize, sink:usize){ // max flow
  flow=0;              // augment while path exists
  loop {
    pushed=bfs_path(source,sink);
    if pushed==0 {break}
    flow+=pushed;
    v=sink;
    while v!=source {
      u=parent[v].unwrap();
      cap[u][v]-=pushed; cap[v][u]+=pushed;
      v=u
    }
  }
  flow
}
```

Ford-Fulkerson integer caps: $O(E|f^*|)$**. Edmonds-Karp BFS:** $O(VE^2)$**. Final residual graph gives min cut by residual reachability from** s.

Max-Flow Min-Cut / Reductions

$$\max |f| = \min_{s \in S, t \notin S} c(S, V \setminus S).$$

Optimality certificate: flow value F **and cut capacity** F.
Bipartite matching: source→ L **cap 1;** $L \to R$ **cap 1 for original edges;** $R \to$**sink cap 1. Flow-1** $L \to R$ **edges are matched pairs.**

```
source=0; left0=1; right0=1+n_left; sink=1+n_left+n_right;
for l in 0..n_left {add_edge(source,left0+1,1)}
for (l,r) in bip_edges {add_edge(left0+1,right0+r,1)}
for r in 0..n_right {add_edge(right0+r,sink,1)}
// flow-1 edge left0+l -> right0+r = matched pair
```

Edge-disjoint paths: all edges cap 1. Vertex-disjoint: split v **into** $v_{\text{in}} \to v_{\text{out}}$ **cap 1; original** $u \to v$ **becomes** $u_{\text{out}} \to v_{\text{in}}$ **cap INF.**

15. Randomized / Probabilistic Analysis

Indicator: $X_A = 1[A], \mathbb{E}[X_A] = \Pr(A)$**. Linearity:**

$$\mathbb{E}\left[\sum_i X_i\right] = \sum_i \mathbb{E}[X_i]$$

No independence needed.
Hiring:

$$\Pr(X_i = 1) = 1/i \Rightarrow \mathbb{E}[\#\text{hires}] = \sum_{i=1}^{n} 1/i = H_n = \Theta(\log n).$$

Hat-check:

$$\Pr(X_i = 1) = 1/n \Rightarrow \mathbb{E}[\#\text{fixed}] = \sum_{i=1}^{n} 1/n = 1.$$

Randomized quicksort: random pivot avoids input-dependent bad order; expected $\Theta(n \log n)$**, worst still** $\Theta(n^2)$.
Birthday: $\Pr$**[all different among** q **into** $m] = \prod_{i=0}^{q-1}(1 - i/m) \approx e^{-q(q-1)/(2m)}$**; for 365 days,** $q = 23$ **gives match prob** $> 1/2$**. Integral bound for decreasing positive** f**:** $\sum_{k=a+1}^{b} f(k) \le \int_a^b f(x)\,dx$.

16. Online Algorithms / Competitive Analysis / Experts

Online: decide before future known. Competitive ratio:

$$\text{ALG}(I) \le r \cdot \text{OPT}(I) + c.$$

Ski rental: rent 1/day, buy B**, unknown** T**.** $\text{OPT}(T) = \min(T, B)$**. Rent until cost** B**, then buy. Ratio 2.**
Secretary: random order; skip first n/e**, take next candidate better than all seen; success probability** $\sim 1/e$**. Prophet: online threshold vs prophet seeing all values.**